

Politecnico di Torino

Department of Electronics and Telecommunications

Master's degree in Mechatronic Engineering



2025/2026

Sensors, Embedded Systems and Algorithms for Service Robotics

Report 1

Date, 10/11/2025

Group n° 212

Francesca Fornasier – S338688

Salvatore Loizzo – S344605

Daniele Torrero – S348210

Maria Teresa Tucci – S349416

1 OBJECTIVE

The aim of this laboratory activity is to learn how to use the Robot Operating System (ROS 2) to create and evaluate advanced navigation and control strategies for mobile robots.

Through these laboratories we must ensure that the robot:

- Can follow a safe route to its target
- Can recognize when an obstacle is nearby and avoid it

This approach will be tested in both simulated (using Gazebo) and real-world environments, and a comparison between them will be provided.

2 DESIGN

We firstly created two nodes inside the packet lab01_pkg we built: **controller** and **localization**.

The **controller** node publishes at frequency of 1 Hz on a topic called /cmd_vel the velocity commands requested by the task, printing on the shell the published message. To do that we exploited the square geometry and we initialize three class variables:

- self.i = 1 #counts seconds needed to come back to the origin position
- self.m = 4 #indicates the perimeter length of the square to trace
- self.c = 1 #indicates the current cycle

We created a callback_function and initialized an empty Twist message. The change of directions must be every self.m/4 seconds. So, we implemented inside the function an if/elif condition comparing self.i with respect to change of direction, to apply 4 different commands to velocity. Every time the function was called (every second) we increase self.i by one and we publish the Twist message. If self.i had a higher value with respect to the perimeter, we had completed a square and the variables are initialized again (self.i=1, self.c+=1, self.m=self.m*self.c).

As far as **localization** node is concerned, class variables describing position are set to 0.0 (starting position = origin): self.x and self.y. In the subscriber callback function we exploited the Twist message coming from the topic /cmd_vel to compute the current position of the robot (adding to the previous position along a specific axis the result of the product between the velocity along that axis and the interval of time dt=1.0). The localization node publishes the information computed on a topic called /Pose (Pose message).

Then, after checking everything worked properly, (using rqt and Plotjuggler), we modified our topology and the codes of the nodes to reset the position if the distance from the origin was greater than 6 meters. The **reset** node is a subscriber of the topic /pose and computes inside the listen_callback function the distance from the origin ($\sqrt{x^2 + y^2}$). If this distance is greater than 6 the node publishes on the topic /reset a Boolean True value. So, at this point, we modified the controller and localization node (adding new ones inside our package). Both became subscriber to the topic /reset. Whenever the bool value they listen inside the topic is True, they respectively reset the velocity commands and the position information.

In the 2nd and 3rd laboratory (the code for both labs is condensed in lab03_pkg) we provided the implementation of the “bump and go” behaviour of a ROS2 node, in other words the description of a robot obstacle avoidance algorithm. The robot’s movement is autonomous and based on **LIDAR** (LaserScan) data and **odometry** (the robot’s position and orientation). It works in the following way:

1. The robot moves forward until it encounters an obstacle
2. When an obstacle is too close (closer than a certain distance), it stops moving forward
3. It calculates the most open direction around itself
4. It rotates toward that direction until it is properly oriented
5. Then it resumes moving forward.

The defined class ‘**BumpAndGoController(Node)**’ is the main class, in which publisher and subscriber are defined, to interact with ROS2.

- **self.publisher** publishes messages of type **Twist** on **/cmd_vel** (used to move the robot: linear and angular velocity)
- **self.laser_data_subscriber** receives **LaserScan** messages from **/scan** (LIDAR data: distances from obstacles around the robot)
- **self.odometry_subscriber** receives **Odometry** messages from **/odom** (used to determine robot’s orientation, i.e. it’s rotation angle)
- **self.timer** periodically calls (every 1 second) the control function **control_loop_callback**.

The node declares other two parameters that can also be set from the terminal or from a YAML file:

- **max_ang**, maximum angular velocity
- **max_lin**, maximum linear velocity
- **min_distance_limit**, minimum security distance from obstacles.

2.1 TASK 1

Function **get_laser_data_callback**:

It’s called every time new data from the LIDAR (**/scan**) is received.

- It reads the LIDAR’s angular parameters (**angle_min**, **angle_max**, **angle_increment**)
- It converts the list of distances into a NumPy array
- It replaces NaN (invalid) and inf (infinite) values with minimum or maximum range values to avoid errors during the mean value computation
- It divides the LIDAR’s field of view into 12 sectors and calculates the average distance in each sector (from the front all the way around the robot)

Using this function, the robot obtains a simplified map of where nearby obstacles are and where there is free space.

2.2 TASK 2

Function **odometry_callback**:

It updates the actual robot orientation (**self.yaw**) every time a message of the type **/odom** is received.

- It extracts the orientation quaternion
- It converts it into Euler angles
- It memorizes only the yaw (the orientation on the horizontal plane).

It is needed to know the orientation of the robot.

2.3 TASK 3

Control loop **control_loop_callback**:

It is executed every second; it checks robot's state and decides movement commands (Twist).

1. Calculation of front distances: it calculates the average distances in four areas in front of the robot (**distFR** front-right, **distR** right, **distFL** front-left, **distL** left). This is used to determine if there is an obstacle ahead.
2. Obstacle detection: if the robot is not already turning (state != 'TURN') and any of those distances is less than distance_limit, it means there is a nearby obstacle and the robot switches its state into 'TURN'.
3. Choosing the free direction: when it enters the 'TURN' mode, the robot analyzes the 12 sectors calculated before, considering only the front ones (about roughly $\pm 120^\circ$), finds the most open direction, calculates the corresponding angle and computes the desired orientation

$$\text{desired_yaw} = \text{current orientation} + \text{angle to rotate}$$
4. Rotation: as long as the current yaw does not match the desired_yaw (within ± 0.15 rad), the robot remains stationary in the linear direction (linear.x = 0.0) and rotates on itself (angular.z positive or negative depending on the direction). When the angular difference (angle_diff) becomes small, the rotation is considered complete and the robot switches into 'FORWARD' mode.
5. Forward movement: in this state linear.x is set to the maximum speed and angular.z = 0.

3 EXPERIMENTS

First, we practiced with turtlesim package, nodes, topics and visualizations tools like rqt and Plotjuggler (following tutorials linked in the laboratory text). We used a launch file for running three different nodes (controller_reset, localization_reset, reset_node) at the same time. However since we were having some troubles in recording data (they were saved in .db3 files, but Plotjuggler couldn't accept this kind of file), we used a separated terminal to record data with the command:

```
ros2 bag record -a -o ~/ros2_ws/bags/session_$(date +%Y%m%d_%H%M%S)
```

In figure 1 is depicted the three terminals operating together, while in the background rosbag command was recording data.

```

ros2 run lab01_pkg controller_reset
[INFO] [1762807811.770809116] [controller_reset]: Publishing: x dot=-1.0, y dot=0.0
[INFO] [1762807812.770652790] [controller_reset]: Publishing: x dot=-1.0, y dot=0.0
[INFO] [1762807813.770632510] [controller_reset]: Publishing: x dot=-1.0, y dot=0.0
[INFO] [1762807814.770602493] [controller_reset]: Publishing: x dot=-1.0, y dot=0.0
[INFO] [1762807815.770757195] [controller_reset]: Publishing: x dot=0.0, y dot=-1.0
[INFO] [1762807816.770774359] [controller_reset]: Publishing: x dot=0.0, y dot=-1.0
[INFO] [1762807817.770665115] [controller_reset]: Publishing: x dot=0.0, y dot=-1.0
[INFO] [1762807818.771261058] [controller_reset]: Completed cycle! Now p=20 and the robot will change the direction every 5.0 seconds
[INFO] [1762807819.770662780] [controller_reset]: Publishing: x dot=1.0, y dot=0.0
[INFO] [1762807820.770959629] [controller_reset]: Publishing: x dot=1.0, y dot=0.0
[INFO] [1762807821.770639951] [controller_reset]: Publishing: x dot=1.0, y dot=0.0
[INFO] [1762807822.770518218] [controller_reset]: Publishing: x dot=1.0, y dot=0.0
[INFO] [1762807823.770704897] [controller_reset]: Publishing: x dot=1.0, y dot=0.0
[INFO] [1762807824.770811454] [controller_reset]: Publishing: x dot=0.0, y dot=1.0
[INFO] [1762807825.770599981] [controller_reset]: Publishing: x dot=0.0, y dot=1.0
[INFO] [1762807826.770800439] [controller_reset]: Publishing: x dot=0.0, y dot=1.0
[INFO] [1762807827.770670954] [controller_reset]: Publishing: x dot=0.0, y dot=1.0
[WARN] [1762807827.783679007] [controller_reset]: Reset received! Restarting sequence.
[INFO] [1762807828.770707623] [controller_reset]: Publishing: x dot=1.0, y dot=0.0

ros2 run lab01_pkg localization_reset
[INFO] [1762807809.782776266] [localization_reset]: Publishing pose: x=4.0, y=3.0, theta=0.0
[INFO] [1762807810.782783718] [localization_reset]: Publishing pose: x=4.0, y=4.0, theta=0.0
[INFO] [1762807811.782609751] [localization_reset]: Publishing pose: x=3.0, y=4.0, theta=0.0
[INFO] [1762807812.782766423] [localization_reset]: Publishing pose: x=2.0, y=4.0, theta=0.0
[INFO] [1762807813.782552035] [localization_reset]: Publishing pose: x=1.0, y=4.0, theta=0.0
[INFO] [1762807814.782651085] [localization_reset]: Publishing pose: x=0.0, y=4.0, theta=0.0
[INFO] [1762807815.782616871] [localization_reset]: Publishing pose: x=0.0, y=3.0, theta=0.0
[INFO] [1762807816.782919534] [localization_reset]: Publishing pose: x=0.0, y=2.0, theta=0.0
[INFO] [1762807817.782614299] [localization_reset]: Publishing pose: x=0.0, y=1.0, theta=0.0
[INFO] [1762807818.782845866] [localization_reset]: Publishing pose: x=0.0, y=0.0, theta=0.0
[INFO] [1762807818.783397390] [localization_reset]: Completed a square!
[INFO] [1762807819.782556299] [localization_reset]: Publishing pose: x=1.0, y=0.0, theta=0.0
[INFO] [1762807820.782616655] [localization_reset]: Publishing pose: x=2.0, y=0.0, theta=0.0
[INFO] [1762807821.782989842] [localization_reset]: Publishing pose: x=3.0, y=0.0, theta=0.0
[INFO] [1762807822.782402957] [localization_reset]: Publishing pose: x=4.0, y=0.0, theta=0.0
[INFO] [1762807823.782776066] [localization_reset]: Publishing pose: x=5.0, y=0.0, theta=0.0
[INFO] [1762807824.782708515] [localization_reset]: Publishing pose: x=5.0, y=1.0, theta=0.0
[INFO] [1762807825.782611697] [localization_reset]: Publishing pose: x=5.0, y=2.0, theta=0.0
[INFO] [1762807826.782692060] [localization_reset]: Publishing pose: x=5.0, y=3.0, theta=0.0
[INFO] [1762807827.782689551] [localization_reset]: Publishing pose: x=5.0, y=4.0, theta=0.0
[WARN] [1762807827.783499213] [localization_reset]: Reset received! Returning to origin.
[INFO] [1762807828.782662772] [localization_reset]: Publishing pose: x=1.0, y=0.0, theta=0.0

ros2 run lab01_pkg reset_node
[WARN] [1762807778.797332209] [reset_node]: Distance > 6 m! Publishing reset=True
[WARN] [1762807827.783315803] [reset_node]: Distance > 6 m! Publishing reset=True

```

Figure 1 - Nodes ran through Launch file

3.1 SIMULATION

In the lab 2 simulation, we initially checked for obstacles by analysing LIDAR data within a $\pm 30^\circ$ range around the robot's direction of motion. We computed the average distance in this sector and stopped the robot if it was below a threshold of 0.5 m. However, we observed that this approach could miss certain obstacles, particularly when the robot approached them at a nearly parallel angle or when the obstacle only partially occupied the scanned area, resulting in an average distance above the threshold.

To improve the accuracy, we expanded the sensing sector to 80° and divided it into four 20° subsectors, computing the average distance for each one to have a more precise and reliable obstacle detection.

In the initial runs, we used the recommended maximum angular velocity of 1.5 rad/s. The robot's orientation was monitored by comparing the desired rotation angle with the current orientation received from odometry messages on the /odom topic, and the robot stops once the desired angle is reached within a tolerance of 0.1 rad.

Since the control_loop_callback method runs at 1 Hz, the robot could overshoot the desired orientation before it was checked, causing it to reverse direction and oscillate around the target angle. To mitigate this issue, we reduced the rotation speed to 0.3 rad/s and increased the angular tolerance to 0.15 rad.

3.2 REAL ROBOT

For the real robot testing, we used the same controller developed in Lab 2, modifying the Quality of Service (QoS) settings of the subscription to the /scan topic to ensure proper operation on real hardware.

Additionally, since it is not guaranteed that the real sensor provide exactly 360 values in the ranges field, we used NumPy's split() method instead of reshape() to divide the space around the robot into sectors, as reshape() requires the array size to be perfectly divisible by the desired shape. We also adjusted the index computation for the relevant elements as:

$$\text{int}((\text{np.deg2rad}(x) - \text{self.angle_min}) / \text{self.angle_increment})$$

where x corresponds to the index used in the previous implementation (assuming 360 elements in the ranges field), and `self.angle_min` and `self.angle_increment` are parameters obtained from the `laser_msg` published on the `/scan` topic.

After deploying and running the controller on the real robot, we observed that it behaved consistently with the simulation results: the robot stopped when approaching an obstacle and rotated toward the most open direction within approximately a $\pm 120^\circ$ range around the direction of its forward motion.

Since the testing environment was relatively narrow, we conducted a second experiment, reducing the obstacle detection threshold to 0.4 m. This allowed the robot to approach obstacles more closely and reduced the frequency of unnecessary stops for direction correction.

4 RESULTS AND DISCUSSION

As far as the laboratory 1 is concerned the data acquired by Plotjuggler confirmed that the nodes worked properly. In Figure 2 and 3 are depicted respectively the plot on Plotjuggler of x-y pose of the normal controller and of the controller with reset node active. The plots confirmed what expected and requested by the task. It was a first exercise useful to get in practice with ROS2 basic concepts of publisher/subscriber, topics and launch file.

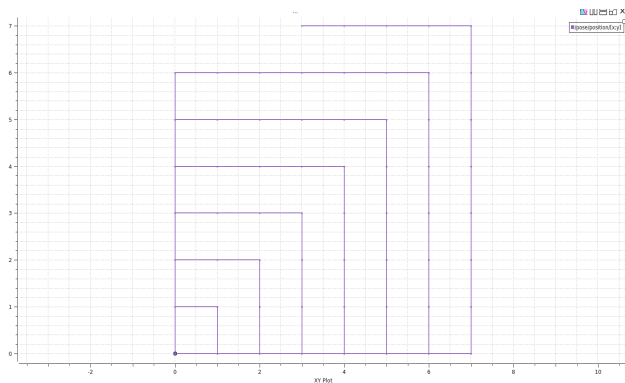


Figure 2 –Plotjuggler x-y Lab1 controller

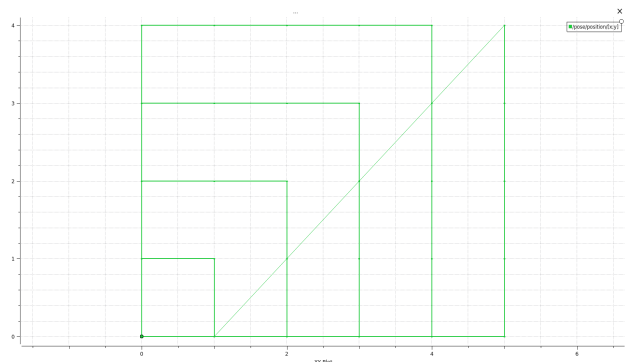


Figure 3 –Plotjuggler x-y Lab1 controller reset

To obtain the RVIZ2 simulation (Figure 4) based on the real data acquired with the real robot we executed on a terminal, after launching `rviz`, the command:

```
ros2 bag play ~/ros2_ws/bags/rosbag2_2025_11_06-18_59_06
```

This command let us play the data exchanged on the topic during the real experiment. In order to obtain permanent traces of the walls and the path, we changed the parameters “Decay Time” of LaserScan display to 1000 on RVIZ2, the “keep” parameter of odometry and some features linked to the shape of the arrow and shaft.

Moreover, we uploaded the data in Plotjuggler and we plotted the x-y data inside /odom topic (.pose.pose.position.x and .pose.pose.position.y) obtaining the path in Figure 2. We did the same process to obtain Figure 3 and Figure 4 (we put the Gazebo picture because in simulation we already had a model of the world).

By comparing Figure 5 and Figure 7, which show the real and simulated robot paths respectively, we can observe an apparent difference in behavior. In the simulation, the robot moves in well-defined straight-line trajectories, while in the real-world test it alternates more frequently between linear and rotational velocity commands. This behavior is mainly due to the narrower corridors in the real environment; because of the tolerance settings (as discussed in the Experiment section), the robot needed to adjust its direction multiple times before finding an orientation that allowed it to move almost parallel to the walls.

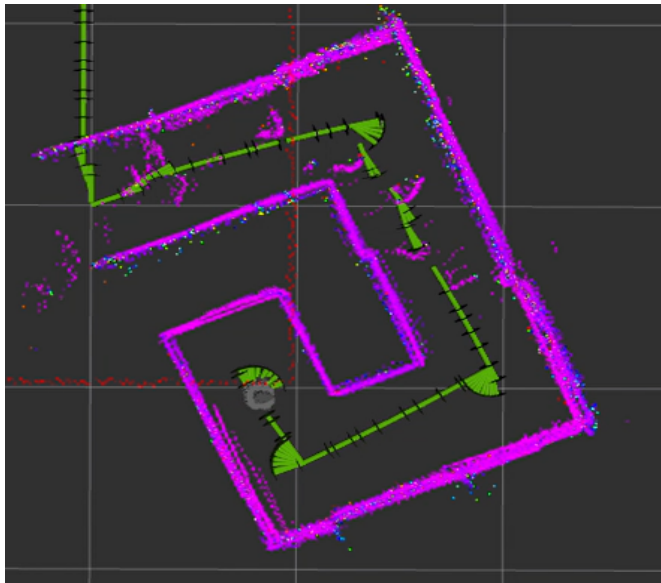


Figure 4 - Simulation on RVIZ2 of real data

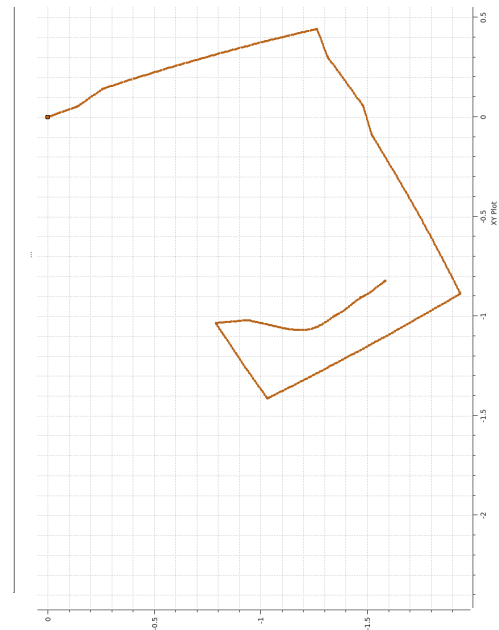


Figure 5 – PLOTJUGGLER position x-y (real data)

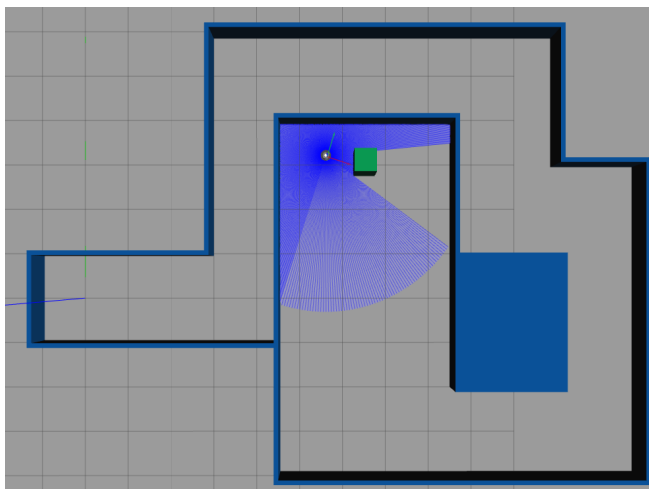


Figure 6 - Simulation on GAZEBO (theorical model)

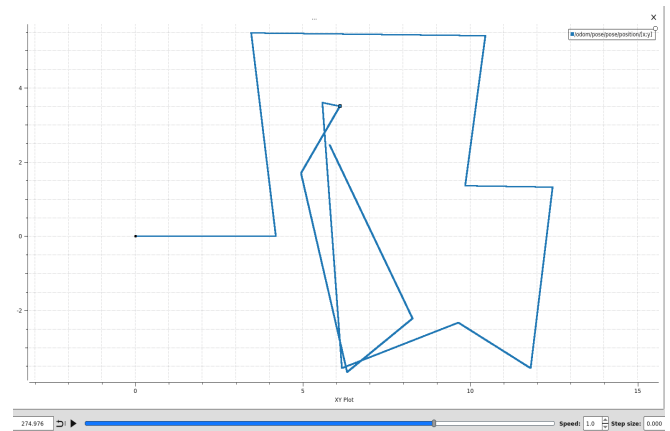


Figure 7 - PLOTJUGGLER position x-y (theorical model)

Comparing Figure 8 and 9 we could notice a certain delay, error, real behaviour of real components (motors) to pass from linear velocity command along x-axis to angular velocity commands along the z-axis. In general there are much more non ideal phenomenon such as ground friction, communication latency, sensors noise. Despite the real implementation the overall behaviour of the robot satisfied the goals requested by the task: collision avoidance in an unknown path, in order to reach a final position.

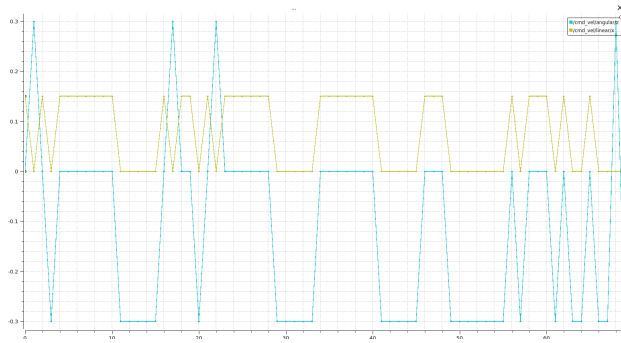


Figure 8 – PLOTJUGGLER linear velocity $x_{\dot{}}$ and angular velocity along z-axis (real model)

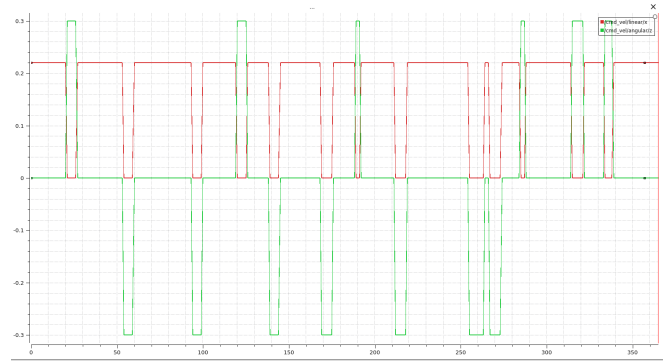


Figure 9 – PLOTJUGGLER linear velocity $x_{\dot{}}$ and angular velocity along z-axis (theoretical model)